

SOC ANALYST PORTFOLIO CASE STUDY

osquery Endpoint Threat Hunting

A repeatable SQL-based hunt for users, processes, listening services, connections, packages, and deleted executables

A controlled localhost listener was detected, attributed to its process and user, independently validated, removed, and confirmed absent in the final state.

Analyst	Ryan Bryant
Exercise date	August 29, 2026
Environment	Authorized VirtualBox SOC home lab
Classification	Portfolio demonstration - no production systems

AUTHORIZED-LAB SAFETY NOTE

All scanning, authentication attempts, firewall actions, and log collection occurred inside a controlled private network owned and operated by the analyst.

Executive Summary

This lab installed osquery 5.23.1 on LINUX-WEBSERVER01 and used targeted SQL queries to establish an endpoint baseline and hunt for conditions relevant to a SOC investigation. The hunt reviewed local users, security processes, Suricata, listening ports, active connections, installed security packages, and processes whose executables were missing from disk. A harmless Python HTTP listener was then launched on 127.0.0.1:8088. osquery identified the listener's PID, executable path, command line, and owning user; curl returned HTTP 200; the process was stopped; and the final listening-port query returned an empty result.

STAKEHOLDER OUTCOME

The hunt converted a broad endpoint question into a reproducible query pack with a complete detect-validate-remove-verify evidence chain.

Objective and Scope

- **Primary objective:** Demonstrate endpoint investigation using structured, repeatable SQL queries rather than ad hoc shell inspection.
- **Hunt coverage:** Users, security tooling, processes, ports, remote connections, packages, and deleted executables.
- **Controlled anomaly:** A Python listener bound only to loopback on TCP 8088.
- **Safety boundary:** The listener used /tmp content and 127.0.0.1, so it was not exposed to the lab network or internet.

Lab Environment

Role	System	Address / Function
Hunted endpoint	LINUX-WEBSERVER01	10.10.10.4 - Ubuntu, Apache, Suricata, Wazuh agent
Hunt engine	osquery 5.23.1	Interactive osqueryi queries with JSON output
Controlled process	python3	127.0.0.1:8088 - temporary HTTP listener
Evidence host	Parrot Security	Private collection, hashing, notes, and archive verification

Hunt Design

Hunt question	Key tables / join	Analytic purpose
Who can log on?	users	Identify local accounts and home directories

Hunt question	Key tables / join	Analytic purpose
Are security tools running?	processes	Confirm or challenge expected defensive processes
What is listening?	listening_ports + processes + users	Attribute ports to executable, command line, and owner
Who is connected remotely?	process_open_sockets + processes	Review active remote endpoints and process context
What security packages exist?	deb_packages	Validate defensive software inventory
Is code running after deletion?	processes where on_disk = 0	Look for execution without a backing file

Controlled Detection

The temporary Python process listened only on loopback. The join across listening_ports, processes, and users returned a single attributable result: 127.0.0.1:8088, PID 5110, /usr/bin/python3.12, the complete command line, and user socstudent.

```
socstudent@linux-webserver01:~$ sudo osqueryi --json "SELECT l.address, l.port, l.protocol, p.pid, p.name, p.path, p.cmdline, u.username FROM listening_ports AS l JOIN processes AS p ON l.pid = p.pid LEFT JOIN users AS u ON p.uid = u.uid WHERE l.port = 8088;" | tee /home/socstudent/osquery-07-controlled-listener-detection.json
[
  {
    "address": "127.0.0.1",
    "cmdline": "python3 -m http.server 8088 --bind 127.0.0.1 --directory /tmp",
    "name": "python3",
    "path": "/usr/bin/python3.12",
    "pid": "5110",
    "port": "8088",
    "protocol": "6",
    "username": "socstudent"
  }
]
```

Figure 1. osquery attributed the controlled listener to PID 5110, python3, /usr/bin/python3.12, and user socstudent.

A separate application-layer check returned HTTP 200, confirming that the listening-port record represented a functioning service rather than stale metadata.

```
socstudent@linux-webserver01:~$ curl -sS -o /dev/null -w "Controlled listener HTTP status: %{http_code}\n" http://127.0.0.1:8088/ | tee /home/socstudent/osquery-07b-controlled-listener-validation.txt
Controlled listener HTTP status: 200
socstudent@linux-webserver01:~$
```

Figure 2. curl independently validated that the controlled localhost listener returned HTTP 200.

Removal and Final-State Verification

The process was stopped by the recorded PID. The same port query was rerun and returned an empty JSON array, proving that the expected state change was visible through the hunt method itself.

```
socstudent@linux-webserver01:~$ kill "$(cat /home/socstudent/osquery-controlled-listener.pid)"
sleep 2
sudo osqueryi --json "SELECT l.address, l.port, p.pid, p.name FROM listening_ports AS l JOIN processes AS p ON l.pid = p
.pid WHERE l.port = 8088;" | tee /home/socstudent/osquery-08-post-containment-verification.json
[1]+  Terminated                  nohup python3 -m http.server 8088 --bind 127.0.0.1 --directory /tmp > /tmp/osquery-lab-htt
p.log 2>&1
[
]
socstudent@linux-webserver01:~$
```

Figure 3. After termination, the port-8088 osquery returned an empty result and the Python process exited.

Evidence Summary

The analyst summary documented the baseline, controlled hunt, assessment, evidence integrity, and conclusion. The preserved package included the query pack, JSON results, listener validation, preinstallation baseline, findings, timestamps, and hash manifest.

```
EXECUTIVE SUMMARY
osquery was used to examine local users, security processes, listening ports, active network connections, installed secu
rity packages, and processes whose executables were missing from disk. A harmless Python HTTP listener was then created
to test detection, validation, and cleanup procedures.

BASELINE FINDINGS
- Expected accounts: root, socstudent, and cowrie.
- Apache, SSH, Suricata, and the Wazuh agent were running from valid on-disk executables.
- Expected services were listening on ports 22, 53, 68, and 80.
- Expected connections existed between 10.10.10.4 and the Wazuh server at 10.10.10.3.
- Required security and system packages were installed.
- No running processes with deleted executables were detected.

CONTROLLED HUNT
- A Python HTTP server was started as user socstudent.
- Process ID: 5110.
- Listener: 127.0.0.1:8088 using TCP.
- Executable: /usr/bin/python3.12.
- osquery successfully associated the listener with its process, command line, executable path, and user.
- An HTTP request returned status 200, confirming the listener was operational.
- The controlled process was terminated.
- A follow-up osquery query returned an empty result, confirming port 8088 was no longer listening.

ASSESSMENT
No unexplained malicious processes, listeners, user accounts, packages, or network connections were identified. The cont
rolled test demonstrated that osquery can identify and attribute an unexpected listening service and verify its removal.

EVIDENCE INTEGRITY
Fifteen collected evidence files were hashed with SHA-256. Every file passed verification.

CONCLUSION
The endpoint matched its expected operational baseline. The hunt successfully demonstrated endpoint visibility, anomaly
identification, process attribution, validation, cleanup, and post-remediation verification.
[user@parrot]--[SOC-Labs/0squery-Endpoint-Hunting]
$
```

Figure 4. The analyst summary records the baseline, controlled hunt, assessment, evidence integrity, and conclusion.

```

[user@parrot]--[~/SOC-Labs/Osquery-Endpoint-Hunting]
$sha256sum -c evidence/SHA256SUMS.txt | tee notes/integrity-verification.txt
evidence/osquery-01-local-users.json: OK
evidence/osquery-02-security-processes.json: OK
evidence/osquery-02b-suricata-process.json: OK
evidence/osquery-03-listening-ports.json: OK
evidence/osquery-04-active-connections.json: OK
evidence/osquery-05-security-packages.json: OK
evidence/osquery-06-deleted-executables.json: OK
evidence/osquery-07-controlled-listener-detection.json: OK
evidence/osquery-07b-controlled-listener-validation.txt: OK
evidence/osquery-08-post-containment-verification.json: OK
evidence/osquery-controlled-listener-http.log: OK
evidence/osquery-controlled-listener.pid: OK
evidence/osquery-evidence-inventory.txt: OK
evidence/osquery-install-verification.json: OK
evidence/osquery-preinstallation-baseline.txt: OK
[user@parrot]--[~/SOC-Labs/Osquery-Endpoint-Hunting]
$

```

Figure 5. SHA-256 verification returned OK across the osquery evidence set.

Verified Results

Measure	Result	Stakeholder meaning
Hunt domains	7	The query pack covered identity, process, network, package, and file-state signals
Controlled listeners	1	The known anomaly was detected without ambiguity
Attribution fields	PID, path, command, user	The result supported immediate triage and ownership
Independent validation	HTTP 200	The listening service was confirmed active
Post-removal state	Empty JSON array	The query demonstrated successful cleanup

Findings and Limitations

Confirmed findings

- osquery exposed endpoint state in structured JSON suitable for repeatable investigation and automation.
- The controlled listener was linked to the expected executable, command line, PID, address, port, and user.
- The same hunt logic proved the listener no longer existed after removal.
- The endpoint's expected defensive services and packages were documented as a baseline.

What was not proven

- The lab did not test scheduled query performance, fleet management, event-based tables, retention, or enterprise query load.
- Loopback-only activity does not model a remotely reachable backdoor or prove malicious intent.
- A clean point-in-time result does not establish continuous absence between query executions.

Recommendations

Priority	Recommendation	Rationale
High	Deploy the validated query pack on a schedule with central result collection.	Converts one-time inspection into continuous visibility.
High	Baseline approved listeners, users, packages, and defensive processes by asset role.	Creates context for distinguishing expected state from anomalies.
Medium	Alert on new externally bound listeners and processes with <code>on_disk = 0</code> .	Prioritizes conditions with stronger threat relevance.
Medium	Enrich results with asset owner, criticality, and change-window data.	Improves triage speed and reduces false escalation.

Employer-Facing Project Summary

This project demonstrates endpoint threat-hunting fundamentals: translating investigative questions into SQL, joining endpoint tables for attribution, validating a finding through a second data source, confirming remediation, and packaging repeatable queries and evidence.

Resume Bullet

PORTFOLIO-READY BULLET

Built an osquery endpoint-hunting pack across seven security domains and validated it by detecting, attributing, independently confirming, removing, and rechecking a controlled localhost listener.

Interview Talking Points

- Why joining `listening_ports`, `processes`, and `users` produced more useful evidence than a port list alone.
- How the controlled listener tested detection quality without creating external exposure.
- Why rerunning the exact query after cleanup provided stronger closure evidence.

Skills Demonstrated

osquery | SQL | Endpoint hunting | Process analysis | Port attribution | JSON evidence | Baseline analysis | Verification

AI and Source Transparency

The analyst personally configured the virtual machines, executed the commands, validated the alerts, interpreted the results, captured the screenshots, and preserved the evidence. Generative AI assisted with command sequencing, troubleshooting suggestions, and documentation structure. Technical claims were checked against direct lab evidence and the official references listed below.

References

- osquery documentation - schema. <https://osquery.io/schema/>
- osquery documentation - SQL introduction. <https://osquery.readthedocs.io/en/stable/introduction/sql/>